

Week4 Skill

Task 1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structure.

```
rishill10@ASUS16X: ~/Skill4 × + ∨  
rishill10@ASUS16X:~$ mkdir Skill4  
rishill10@ASUS16X:~$ cd Skill4  
rishill10@ASUS16X:~/Skill4$ nano task1.c  
rishill10@ASUS16X:~/Skill4$ gcc task1.c -otask1  
rishill10@ASUS16X:~/Skill4$ ./task1  
Enter a command: echo Hello World  
Argument 1: echo  
Argument 2: Hello  
Argument 3: World  
rishill10@ASUS16X:~/Skill4$ |
```

```
GNU nano 8.7.1 task1.c  
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char input[100];  
  
    printf("Enter a command: ");  
    fgets(input, sizeof(input), stdin);  
  
    char *token = strtok(input, " \\t\\n");  
    int count = 1;  
  
    while (token != NULL) {  
        printf("Argument %d: %s\\n", count++, token);  
        token = strtok(NULL, " \\t\\n");  
    }  
  
    return 0;  
}
```

Task 2: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
rishill10@ASUS16X: ~/Skill4
rishill10@ASUS16X:~/Skill4$ nano task2.c
rishill10@ASUS16X:~/Skill4$ gcc task2.c -o task2
rishill10@ASUS16X:~/Skill4$ ./task2
Enter text: echo "Hello World"
echo "Hello World"
rishill10@ASUS16X:~/Skill4$ |
```

```
GNU nano 8.7.1 task2.c *
#include <stdio.h>

int main() {
    char str[100];

    printf("Enter text: ");
    fgets(str, sizeof(str), stdin);

    printf("%s", str);

    return 0;
}|
```

Task 3: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
rishill10@ASUS16X:~/Skill4$ echo "Hello $USER"
Hello rishill10
rishill10@ASUS16X:~/Skill4$ |
```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
rishill10@ASUS16X: ~/Skill4 × + v
rishill10@ASUS16X:~/Skill4$ mkdir My\Skill4
rishill10@ASUS16X:~/Skill4$ touch task4.txt
rishill10@ASUS16X:~/Skill4$ echo \$HOME
$HOME
rishill10@ASUS16X:~/Skill4$ |
```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
rishill10@ASUS16X: ~/Skill4 × + v - □ ×
rishill10@ASUS16X:~/Skill4$ nano task5.c
rishill10@ASUS16X:~/Skill4$ gcc task5.c -o task5
rishill10@ASUS16X:~/Skill4$ ./task5
Parent PID: 1295
Child PID: 1296
MySkill4 task1 task1.c task2 task2.c task4.txt task5 task5.c
Child process completed.
rishill10@ASUS16X:~/Skill4$ |
```

```
rishill10@ASUS16X: ~/Skill4 × + v
GNU nano 8.7.1 task5.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) { // Child process
        printf("Child PID: %d\n", getpid());
        char *args[] = {"ls", NULL}; // Argument array for execvp
        execvp("ls", args);
        perror("execvp"); // Only reached if exec fails
        return 1;
    } else { // Parent process
        printf("Parent PID: %d\n", getpid());
        wait(NULL); // Wait for child to finish
        printf("Child process completed.\n");
    }

    return 0;
}
```